

Platform Rebuild Proposal — Executive Summary

Purpose: Convince management to approve an incremental platform rebuild that reduces version sprawl, speeds delivery, and makes client customization and integrations sustainable.

Audience: Executive leadership, product, engineering, professional services, support.

Date: June 2026

1. Core recommendation

Rebuild the platform **incrementally** using a **strangler-fig** approach: a single canonical core with **versioned extension and integration packs** — not a big-bang rewrite and not continued forking per client.

Ask:

- Charter a **Platform 2.0** program with dedicated capacity (not 20% time).
- Approve a **3–6 month foundation + pilot** phase with measurable exit criteria.
- Adopt policy: **no new git submodules, no new core forks, no new SaveChanges workflow handlers.**

2. The problem (in business terms)

We do not have a feature problem. We have a **variant problem**.

| Symptom | Business impact |

|-----|-----|

| Simple changes take weeks | Missed commitments, unhappy customers |

| Regressions in unrelated areas | Support load, emergency releases |

| Integrations behave differently per client | High PS cost, slow onboarding |

| Cloud + on-prem + customized builds | Too many versions to test and support |

| “Which version is running?” | Long incidents, risky upgrades |

| New hires slow to contribute | Delivery does not scale with headcount |

One line: *We are paying a growing tax on every feature because customization and integrations were implemented as compile-time forks, not runtime configuration.*

3. Root technical causes

1. **Git submodules and client-specific code branches** multiply release combinations.

2. **Class inheritance for services** (`ClientXService : BaseService`) spreads logic across opaque override trees.
3. **Per-client data model forks** make migrations, APIs, and reporting unsustainable.
4. **EF `SaveChanges` change handlers** implement hidden workflows (legacy of Access DB + stored procedures).
5. **Hangfire jobs** duplicate and defer logic that should be explicit orchestration.
6. **Weak domain boundaries** — WBS, planning, hours, imports, and integrations are coupled.

This is normal debt from years of shipping under pressure. It is not a people problem.

4. Strategic goal: one platform, many configurations

Design explicitly for three dimensions:

Dimension	Target
Deployment	Same product artifact for cloud SaaS and on-prem (profile-driven config).
Integration	Connector packs (SAP, PLM, HR, file, etc.) — not core forks.
Customization	Configuration, policy hooks, and extension actors — not submodules.

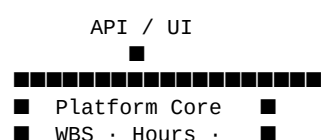
Success criterion: A new client integration or customization ships as a **versioned pack** enabled per tenant, not as a separate product build.

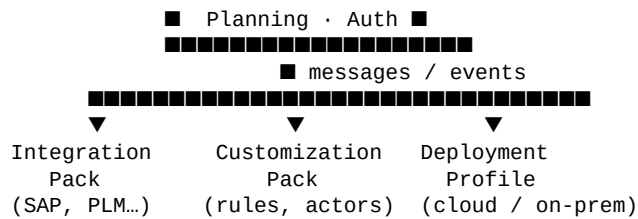
5. Why the current customization model fails

Approach	Long-term result
Git submodules per client/integration	Version matrix explosion, merge hell
Subclassing services	Fragile base classes, hard-to-find overrides
Custom EF models per client	Forked migrations, APIs, reports, upgrades
`if (tenant == X)` in core	Accidental complexity, untestable branches

Replacement principle: *Professional services may customize behavior, not the core release artifact.*

6. Target architecture: platform core + packs





Core owns: canonical domain, invariants, APIs, persistence boundaries, external ID registry.

Packs own: client- or system-specific mapping, validation, enrichment, connectors, and policy steps.

7. Akka.NET: why it fits (orchestration, not fashion)

Akka.NET is proposed because our hardest problems are **workflows**, not CRUD:

- Import and export pipelines
- Integration retries and failure isolation
- Long-running recalculations (planning, rollups)
- Tenant-specific processing steps

Actors vs. inheritance/submodules

| Legacy | Akka approach |

|-----|-----|

| `AcmeImportService : ImportService` | `ImportPipeline → [MapActor, AcmeRulesActor, PersistActor]` |

| Client submodule | Tenant profile registers actor pack |

| Hidden handler chain | Explicit message pipeline with correlation ID |

Actors vs. SaveChanges handlers + Hangfire

| Legacy pattern | Problem | Akka replacement |

|-----|-----|

| EF change handlers on save | Implicit order, nested saves, hard to test | Command → orchestrator actor → persist actor |

| Hangfire jobs with business logic | Two workflow systems, duplicated rules | Supervised connector/workflow actors |

| Scheduled sync | Job-centric observability | Scheduler triggers actor message (Hangfire optional as clock during migration) |

Rule: `SaveChanges` is for **persistence**, not for **business process orchestration**.

Workflow model:

Command (ImportProject, BookHours, RecalculatePlan)

- Orchestrator actor (DataManager, IntegrationRouter, PlanningCoordinator)
- Persist actor (sole EF boundary for that workflow)
- Integration / rollup / notification actors
- Domain events (ImportPersisted, PlanRecalculated)

What Akka is NOT for

- Simple CRUD endpoints
- Sprinkling `DbContext` in every actor
- Replacing a disciplined domain model

Use actors at **boundaries**; keep core business rules in plain, tested modules.

8. Data model strategy (avoid per-client schema forks)

Actors do not justify per-client OLTP schemas.

1. **One canonical schema** for all customers.
2. **Governed extension data** for rare attributes (registry + validation).
3. **External ID mapping** — source systems map to internal entities without alternate keys.
4. **Read models / projections** for client-specific reporting.

Customize **behavior and mappings**, not the physics of the database except through extension points.

9. Cloud and on-prem: deployment profiles, not forks

Concern	Cloud	On-prem
-----	-----	-----
Identity	Hosted SSO	AD / LDAP / SAML
Messaging	Managed queue	Local queue / outbox
Storage	Object store	Local / customer storage
Updates	Continuous	Customer-controlled bundles
Offline	N/A	File-based import/export first-class

Same binary; differences are **configuration and enabled packs**.

10. Proof of concept (internal evidence)

An internal POC already demonstrates the direction:

- Canonical import model with **external IDs** and idempotent upsert
- **Actor-orchestrated persistence** (data manager hierarchy) — not SaveChanges side effects
- **Template-first persist ordering** among siblings
- **Planning engine** with FS / SS / FF / SF dependencies and lag
- **Domain value types** (`Hours`, `DurationDays`, `PersonName`, `ScheduleDate`) protecting business semantics
- **Automated tests** on scheduling, progress, and import behavior

Message to leadership: We are not betting on a theory. We are asking to scale an approach that already produced working vertical slices.

11. Phased migration plan

Phase 0 — Discovery (4–6 weeks)

- Inventory integrations, submodules, handlers, Hangfire jobs, and schema forks.
- Define parity checklist and tenant pack model.
- **Exit:** Agreed domain order and success metrics.

Phase 1 — Foundation (8–12 weeks)

- Shared contracts, actor registry, tenant profiles, CI, observability.
- Policy: no new handlers/submodules.
- **Exit:** One read or write path on new stack in staging.

Phase 2 — First production domain (12–16 weeks)

- Pilot: import/WBS for one client (cloud or on-prem).
- Retire one submodule or handler family.
- **Exit:** Pilot live; rollback documented.

Phase 3 — Expand

- Planning, hours, additional connectors.
- Published **compatibility matrix**: `Core × Integration packs × Customization packs × Deployment profile`.

Handler / job migration order

1. Freeze new SaveChanges handlers and logic-heavy Hangfire jobs.
2. Classify existing workflows (rollup, integration, batch, notification).

3. Migrate **import** first (POC exists).
4. Migrate planning recalc, hours rollups, ERP/export connectors.
5. Delete legacy handlers when parity tests pass.

Hangfire may remain temporarily as a **scheduler trigger** only.

12. Metrics

Metric	Why
Active core branches / client forks	Version sprawl
Lead time (idea → production)	Speed
Change failure rate	Quality
Time to add integration	PS margin
Support tickets “unknown version”	Operational pain
On-prem upgrade completion rate	Enterprise risk
% custom work delivered as packs vs forks	Program adoption

13. Risks and mitigations

Risk	Mitigation
Two systems forever	Domain sunset dates; strangler per module
Akka learning curve	Training, narrow actor boundaries, message contract tests
Debugging complexity	Correlation IDs on every workflow (session / trace ID)
Feature starvation	70% platform / 30% critical customer work via adapters
Big-bang temptation	Explicit anti-big-bang charter; pilot-first

14. Options for decision-makers

Option	Description	Verdict
A. Status quo	Keep patching handlers, jobs, submodules	Cheapest now; cost accelerates
B. Incremental rebuild (recommended)	Core + actor/integration/customization packs	Balanced risk; continuous delivery

15. Executive summary (paste-ready)

Our product's complexity is driven less by feature count than by **years of client-specific customizations and integrations**, delivered through git submodules, service inheritance, customized data models, EF change handlers, and Hangfire jobs — patterns inherited from an earlier Access/stored-procedure architecture. Combined with cloud and on-prem delivery, this created too many de facto product versions to test, upgrade, and support.

The proposed rebuild shifts to a **platform-and-packs architecture**: a stable canonical core; integrations as connector packs; customizations as tenant-configured actor pipelines and policy hooks; and deployment profiles for cloud versus on-prem from the **same build**. Long-running and reactive work moves from implicit SaveChanges side effects and background jobs into **explicit, supervised Akka.NET workflows** with clear message contracts.

This restores predictable delivery, reduces support matrix size, protects professional services margin, and gives customers a credible upgrade path without forking the product for each engagement.

16. Decision requested

1. Approve **Platform 2.0** program charter and dedicated team.
2. Fund **discovery + foundation + one pilot domain**.
3. Enforce **no new submodules / handler workflows**.
4. Name **executive sponsor** (engineering + product + commercial).
5. Select **pilot customer** or internal flagship project for first cutover.

Document generated from engineering strategy discussions. Internal use.